23. ROS2 TF2 coordinate transformation

> **pip installation note:** Under PEP 668, Ubuntu 24.04 marks the system Python as "externally managed", so installing directly with pip may report `externally-managed-environment`. `--break-system-packages` allows pip to install packages in that environment; a virtual environment usually does not need this parameter, but this tutorial keeps it so that the commands stay consistent.

1. Introduction to TF2

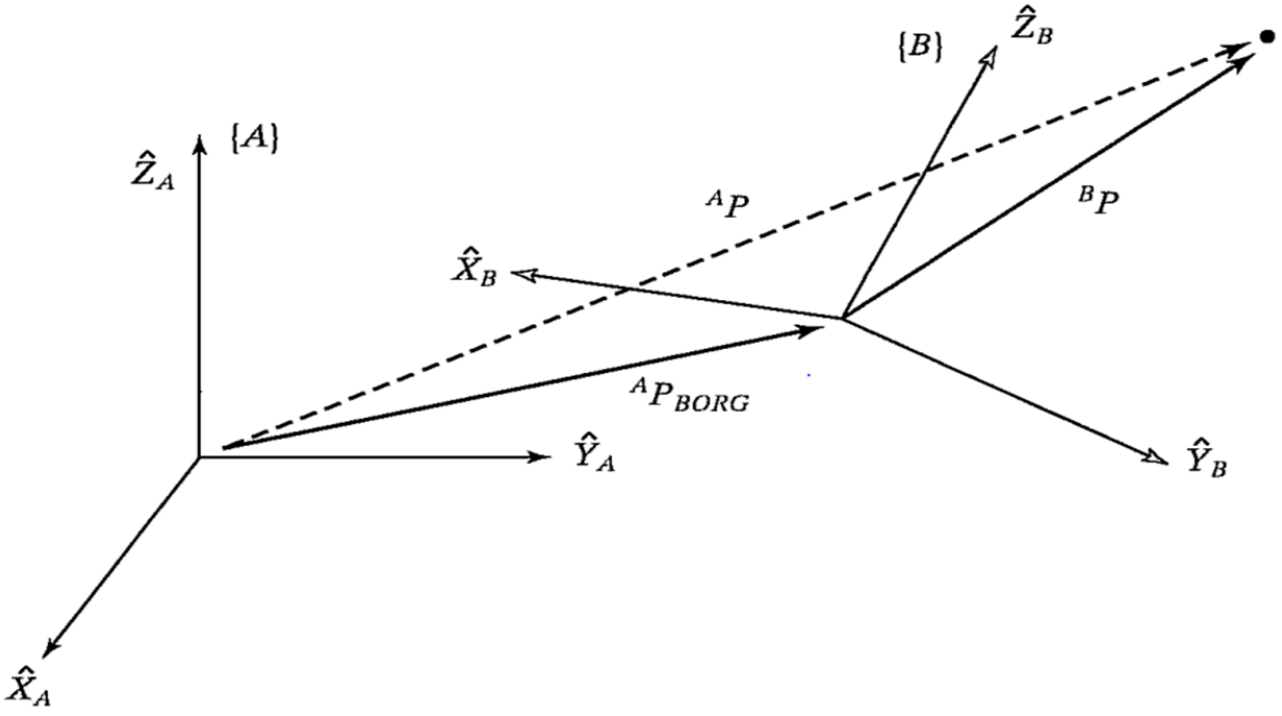
Coordinate systems are a concept we are very familiar with, and they are also an important foundation in robotics. In a complete robotic system there are many coordinate systems, and how should the positional relationships between these coordinate systems be managed? ROS provides us with a magic tool for managing coordinate systems: TF2

TF system reference: [tf: The transform library | IEEE Conference Publication | IEEE Xplore]

2. Coordinate systems in robotics

In a mobile robot system, coordinate systems are just as important. For example, the center point of a mobile robot is the base coordinate system Base Link, the position where the radar is located is called the radar coordinate system laser link, and when the robot moves, the odometry accumulates the position. The reference frame of this position is called the odometry coordinate system odom, and the odometry itself has accumulated errors and drift. The reference frame of the absolute position is called the map coordinate system map.

The relationships between the layers of coordinate systems are complex. Some of them are relatively fixed, while others keep changing. Even seemingly simple coordinate systems become complex within a spatial range, so a good coordinate system management system becomes extremely important.



The basic theory of the coordinate system transformation relationship is explained in every robotics textbook. It can be broken down into **two parts, translation and rotation**, and it is described by a 4x4 matrix. When you draw a coordinate system in space, the transformation relationship between the two is actually the mathematical description of a vector.

The underlying principle of the TF function in ROS is the encapsulation of these mathematical transformations. For detailed theoretical knowledge you can refer to the robotics textbooks; we mainly explain how to use the TF coordinate management system.

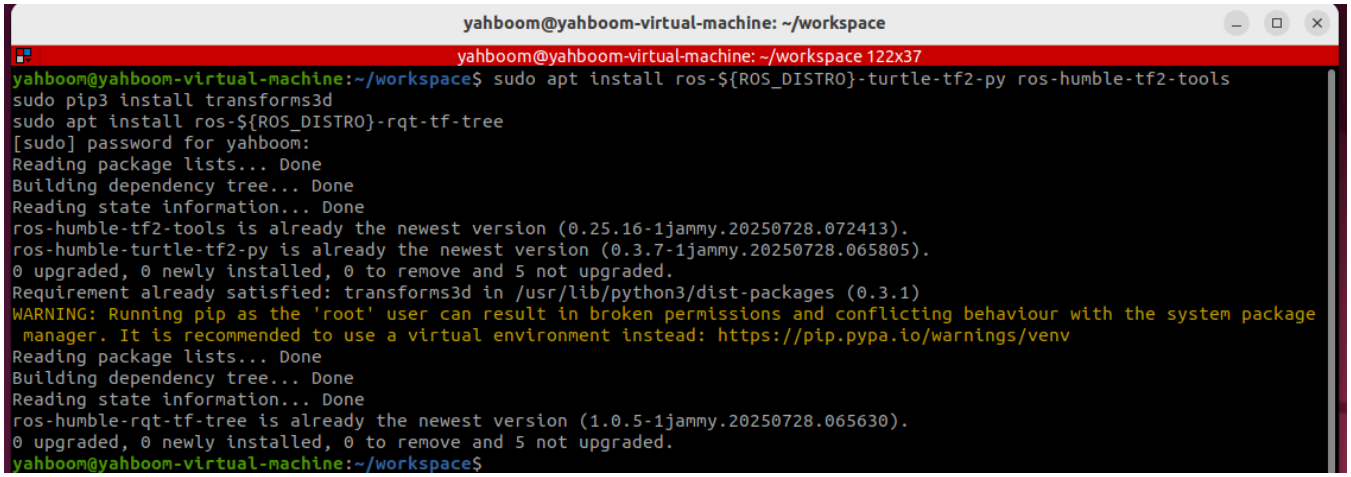
3. TF command line operations

Let's first use the example of two little turtles to understand a robot following algorithm based on coordinate systems. **For the convenience of the demonstration, it is best to perform this lesson in a virtual machine**

3.1. Installing the relevant tools

This example requires us to install the corresponding function packages first: the tf turtle simulator example and the tf tree visualization tool

```
sudo apt install ros-${ROS_DISTRO}-turtle-tf2-py ros-${ROS_DISTRO}-tf2-tools
sudo pip3 install tf_transforms --break-system-packages
sudo apt install ros-${ROS_DISTRO}-rqt-tf-tree
```

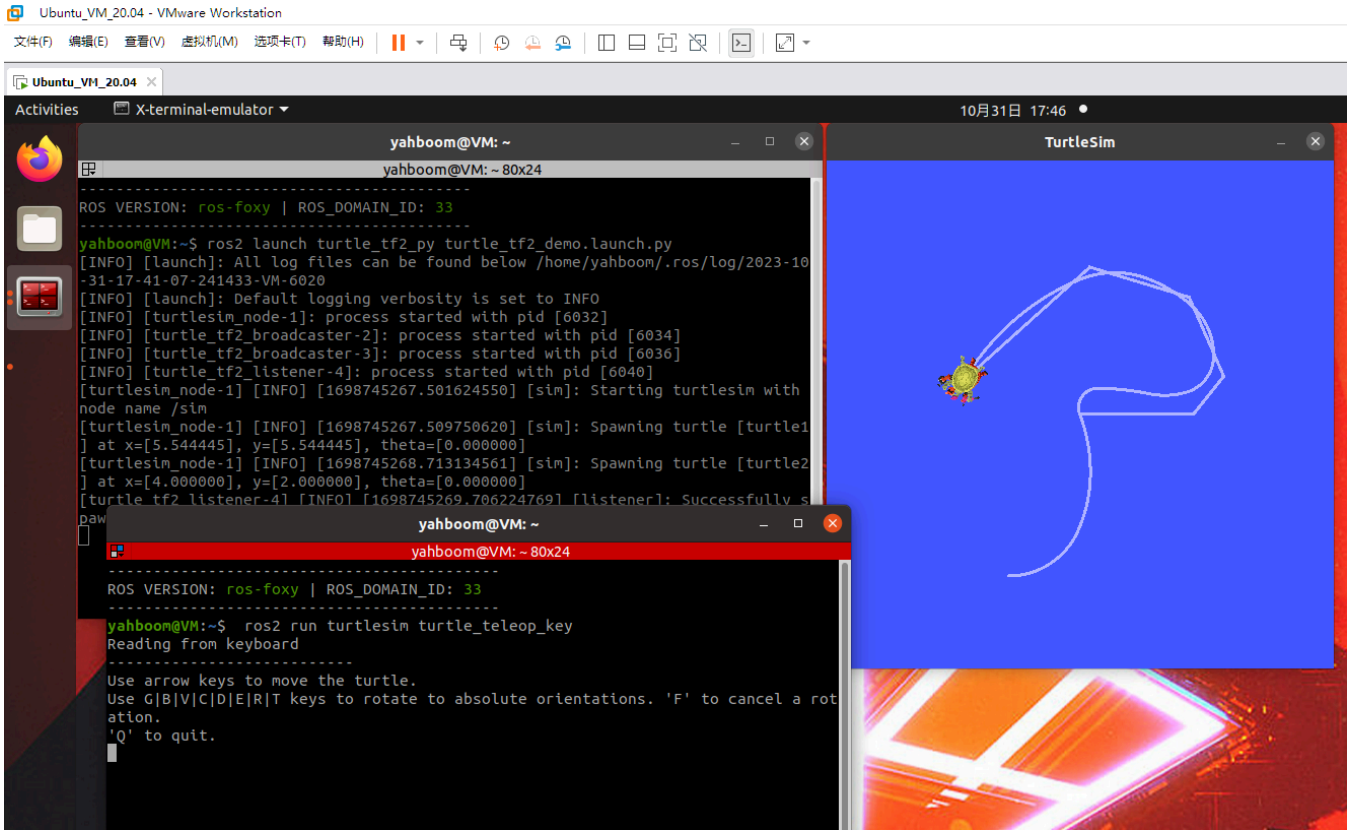


3.2. Starting up

Then you can start it with a launch file. Afterwards we can control one of the little turtles, and the other little turtle will automatically follow it.

```
ros2 launch turtle_tf2_py turtle_tf2_demo.launch.py
ros2 run turtlesim turtle_teleop_key
```

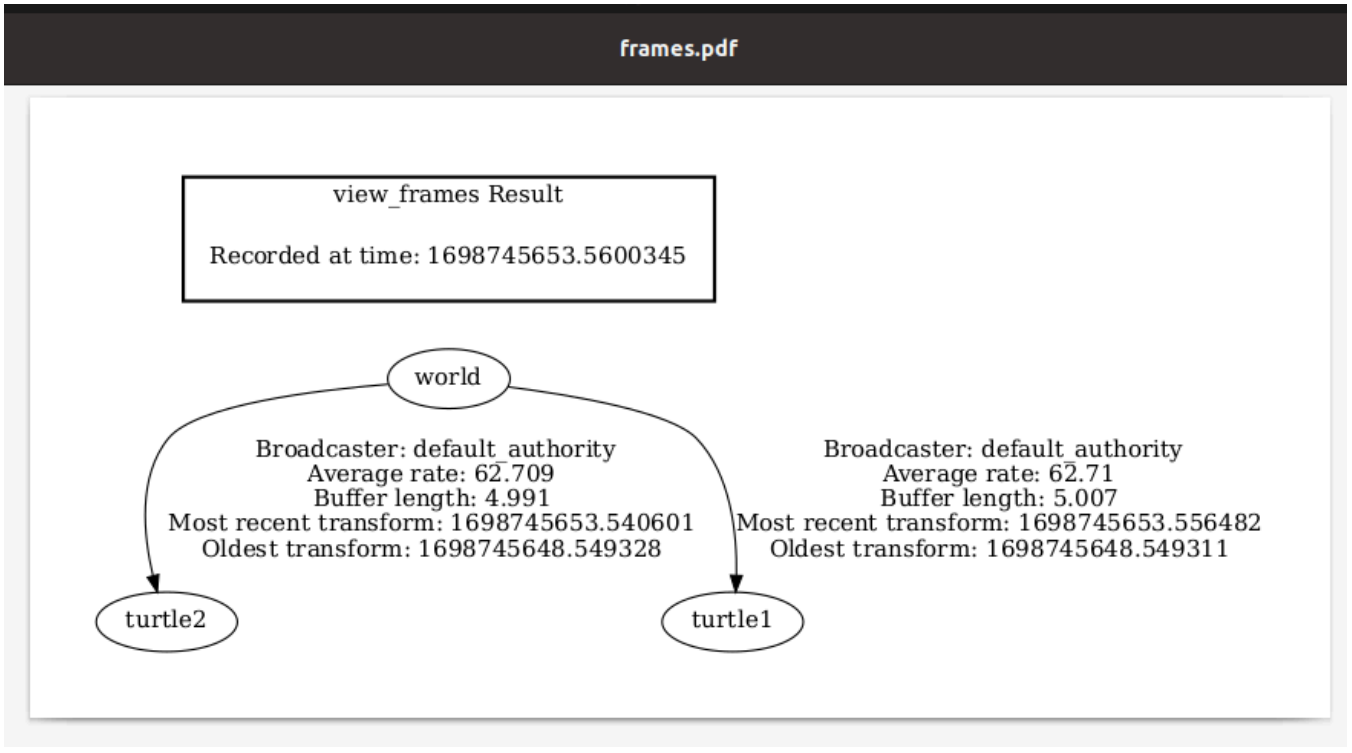
When we control the movement of one turtle, the other turtle will follow it as well.



3.3. Viewing the TF tree

```
ros2 run rqt_tf_tree rqt_tf_tree
```

You can see the TF transformation tree in the rqt window

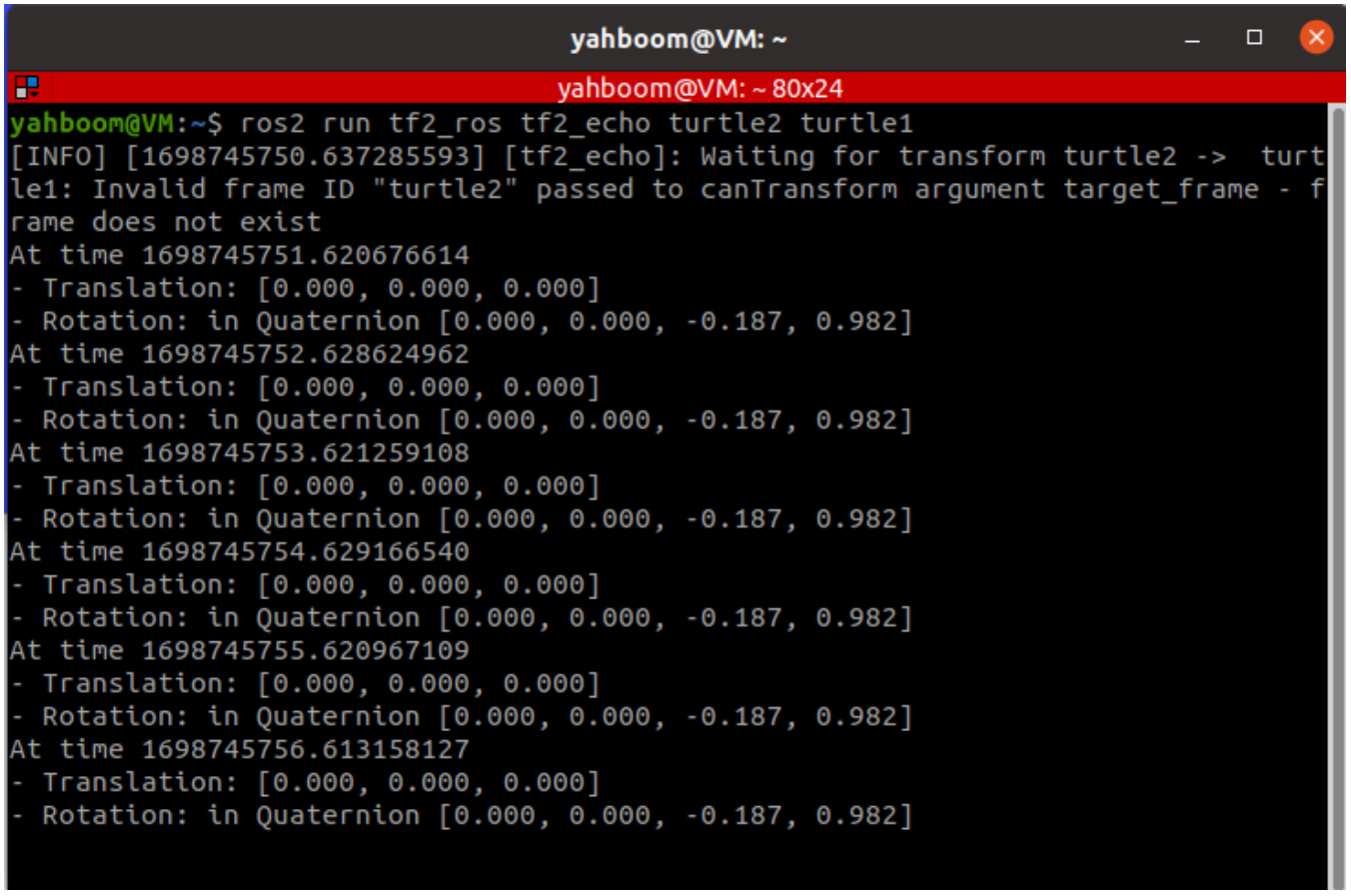


3.4. Querying the coordinate transformation information

Just seeing the structure of the coordinate systems is not enough. If we want to know the specific relationship between two coordinate systems, we can check it with the tf2_echo tool:

```
ros2 run tf2_ros tf2_echo turtle2 turtle1
```

After it runs successfully, the terminal keeps printing the transformation values of the coordinate systems

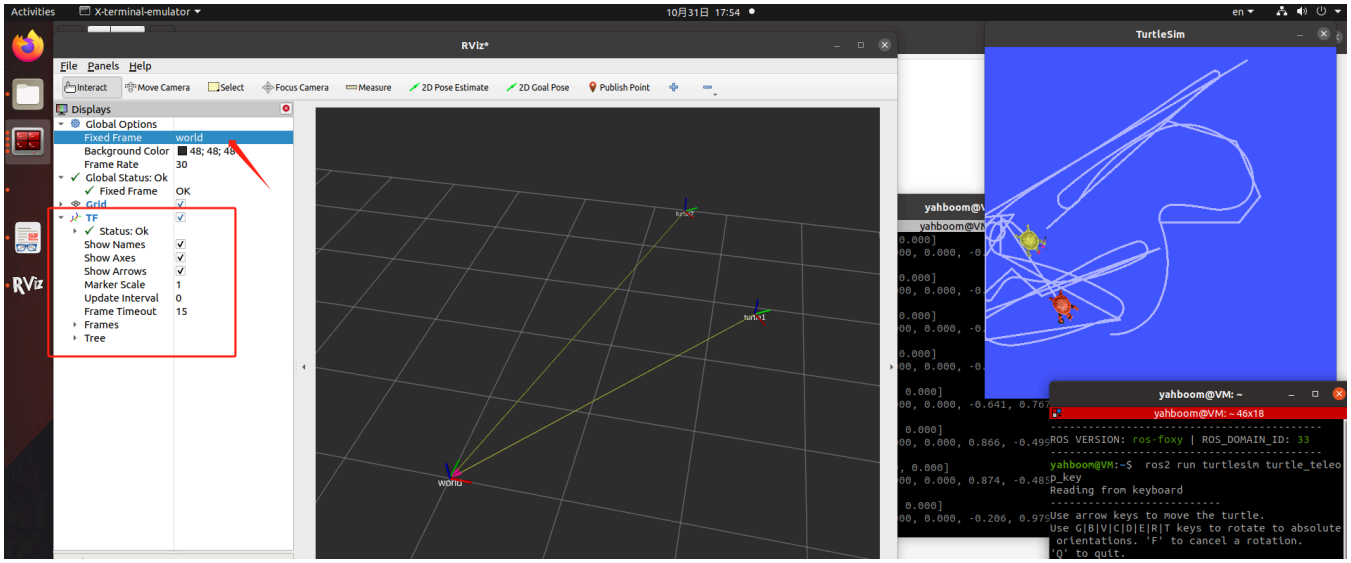


3.5. Coordinate system visualization

- Run rviz2 and then add the TF display plugin

```
rviz2
```

Set the reference coordinate system in rivz2 to: world, add the TF display, and then make the little turtle move. The coordinate axes in Rviz will start to move; isn't this more intuitive!



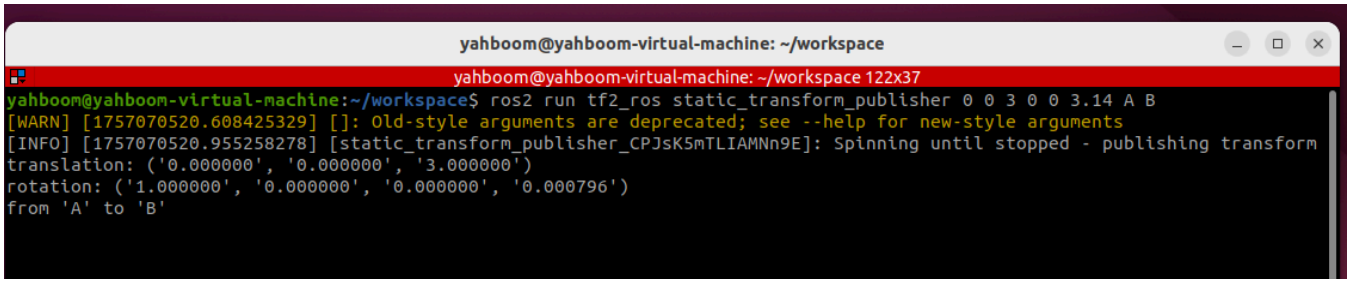
4. Static coordinate transformation

A static coordinate transformation means that the relative position between two coordinate systems is fixed. For example, the position between the radar and base_link is fixed.

Example: **For the convenience of the demonstration, it is best to perform this lesson in a virtual machine**

4.1. Publishing the pose from A to B

```
ros2 run tf2_ros static_transform_publisher 0 0 3 0 0 3.14 A B
```



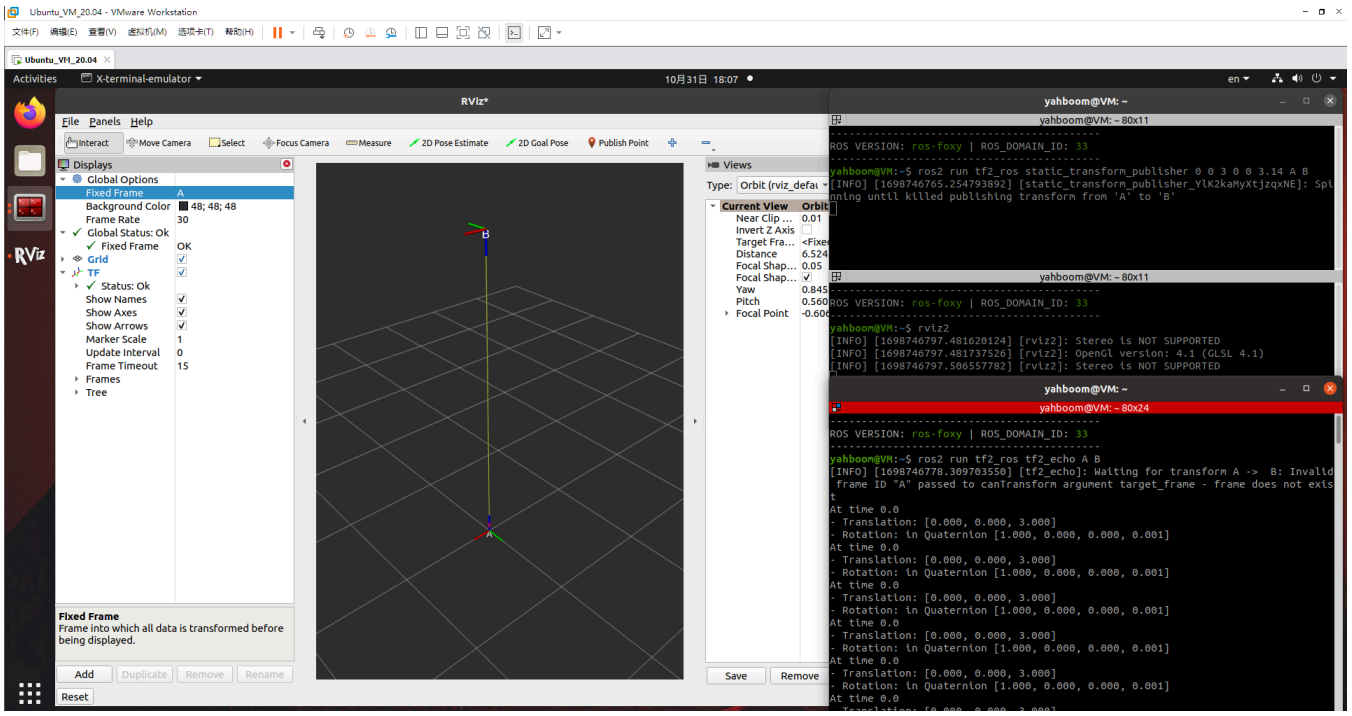
4.2. Monitoring/getting the TF relationship

```
ros2 run tf2_ros tf2_echo A B
```

4.3. rivz visualization

- Run rviz2 and then add the TF display plugin

```
rviz2
```



5. Case introduction

In the previous lesson we explained the TF relationship in the little turtle following example provided by the system. In this lesson we implement this function ourselves.

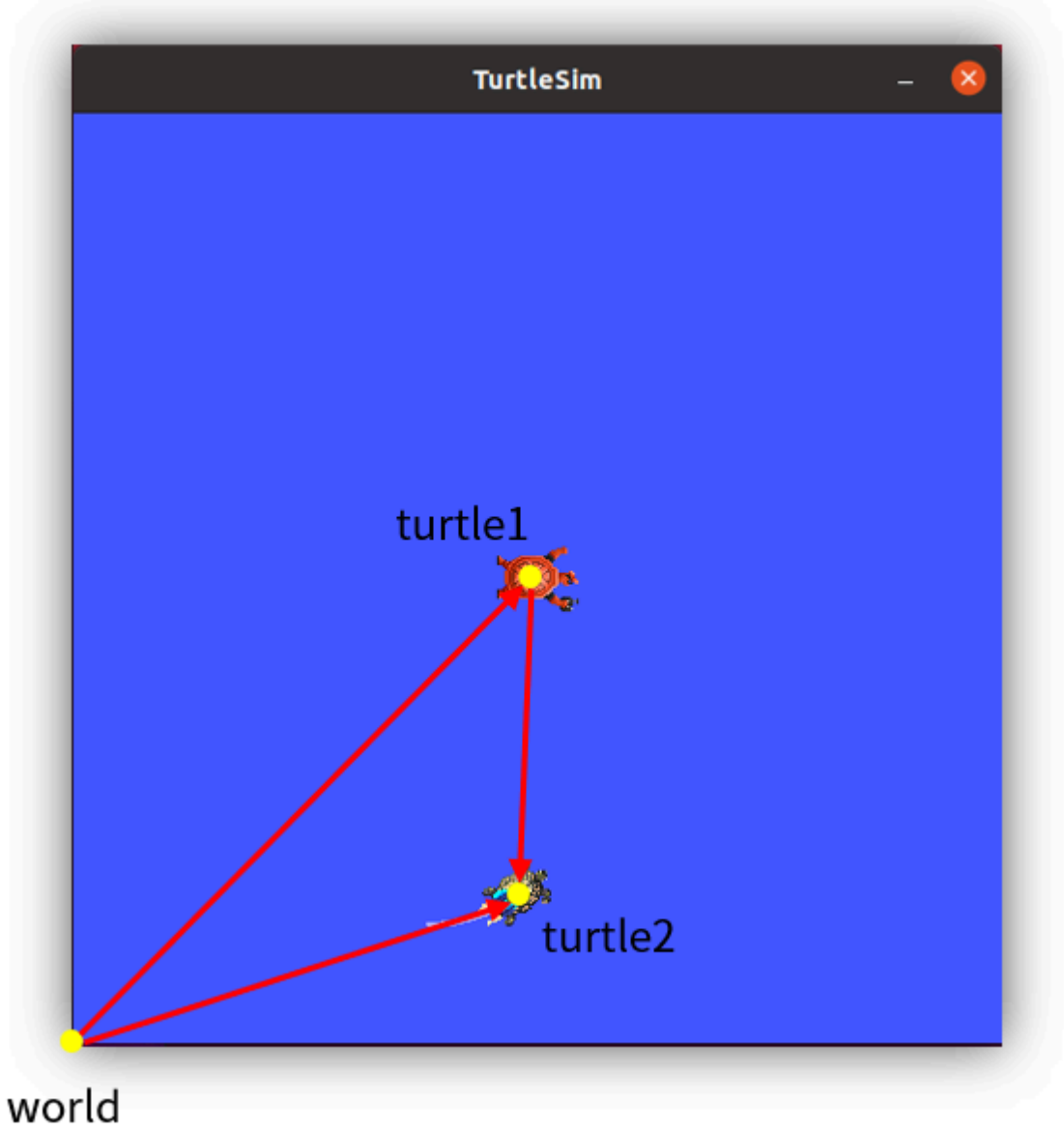
Course content:

- Implement the little turtle following case through programming
- Master the implementation of a dynamic broadcaster through programming
- Master the implementation of the coordinate transformation between the monitored coordinate systems through programming
- Master the use of PID control to convert physical quantities (distance, angle) into speed control variables

Advanced content:

- Understand the ability of the TF system to transform coordinate systems across the time dimension

6. Analysis of the implementation principle of the turtle following case



In the simulator with two turtles, we can define three coordinate systems. For example, the global reference frame of the simulator is called world, and the turtle1 and turtle2 coordinate systems are located at the center points of the two turtles. In this way, the relative position of turtle1 and the world coordinate system represents the position of turtle 1, and turtle 2 works in the same way.

To make turtle 2 move toward turtle 1, we draw a connecting line between the two and add an arrow. Does it remind you of the vector calculation you learned in high school? The way we describe a coordinate transformation is with vectors, so in this following example TF can solve the problem perfectly.

The length of a vector represents the distance and the direction represents the angle. With the distance and the angle, we can set an arbitrary time and then calculate the speed. Then we encapsulate and publish the speed topic, and turtle 2 can start to move.

So the core of this example is to perform the vector calculation through the coordinate system. Since the two turtles keep moving, the vector must also be calculated at a certain period, and this requires the dynamic broadcast and monitoring of TF.

7. Creating a new function package

1. Create a new function package in the src directory of the workspace to store our files

```
ros2 pkg create pkg_tf --build-type ament_python --dependencies rclpy --node-name
turtle_tf_broadcaster
```

After running the command above, the pkg_tf function package is created, and a turtle_tf_broadcaster node is created at the same time, with the relevant configuration files already configured. Add the following code to the [turtle_tf_broadcaster.py] file:

```
import rclpy                                    # ROS2 Python interface library
from rclpy.node import Node                     # ROS2 node class
from geometry_msgs.msg import TransformStamped  # Coordinate transformation message
import tf_transformations                       # TF coordinate transformation library
from tf2_ros import TransformBroadcaster        # TF coordinate transformation
broadcaster
from turtlesim.msg import Pose                  # turtlesim little turtle position
message

class TurtleTFBroadcaster(Node):

    def __init__(self, name):
        super().__init__(name)                              # Initialize the parent class
of the ROS2 node

        self.declare_parameter('turtlename', 'turtle')      # Create a parameter for the
turtle name
        self.turtlename = self.get_parameter(               # The externally set parameter
value takes precedence, otherwise the default value is used
            'turtlename').get_parameter_value().string_value

        self.tf_broadcaster = TransformBroadcaster(self)    # Create and initialize a TF
coordinate transformation broadcast object

        self.subscription = self.create_subscription(       # Create a subscriber to
subscribe to the turtle position messages
            Pose,
            f'/{self.turtlename}/pose',                     # Use the turtle name obtained
from the parameter
            self.turtle_pose_callback, 1)

    def turtle_pose_callback(self, msg):                            # Create a callback
function that handles the turtle position messages and converts the position messages into
coordinate transformations
        transform = TransformStamped()                              # Create a coordinate
transformation message object

        transform.header.stamp = self.get_clock().now().to_msg()    # Set the timestamp of
the coordinate transformation message
        transform.header.frame_id = 'world'                         # Set the source
coordinate system of the coordinate transformation
```

```
        transform.child_frame_id = self.turtlename                    # Set the target
coordinate system of the coordinate transformation
        transform.transform.translation.x = msg.x                     # Set the translation
along the X, Y and Z directions in the coordinate transformation
        transform.transform.translation.y = msg.y
        transform.transform.translation.z = 0.0
        q = tf_transformations.quaternion_from_euler(0, 0, msg.theta) # Convert the Euler
angles to quaternions (roll, pitch, yaw)
        transform.transform.rotation.x = q[0]                         # Set the rotation
along the X, Y and Z directions (quaternions) in the coordinate transformation
        transform.transform.rotation.y = q[1]
        transform.transform.rotation.z = q[2]
        transform.transform.rotation.w = q[3]

        # Send the transformation
        self.tf_broadcaster.sendTransform(transform)     # Broadcast the coordinate
transformation; after the turtle position changes, the coordinate transformation information
will be updated in time

def main(args=None):
    rclpy.init(args=args)                                # Initialize the ROS2 Python
interface
    node = TurtleTFBroadcaster("turtle_tf_broadcaster")  # Create and initialize the ROS2
node object
    rclpy.spin(node)                                     # Loop and wait for ROS2 to exit
    node.destroy_node()                                  # Destroy the node object
    rclpy.shutdown()                                     # Shut down the ROS2 Python
interface
```

2. Next, create a file named [turtle_following.py] in the same directory as turtle_tf_broadcaster.py, and add the following code:

```
import math
import rclpy                                             # ROS2 Python interface library
from rclpy.node import Node                              # ROS2 node class
import tf_transformations                                # TF coordinate transformation
library
from tf2_ros import TransformException                   # TF coordinate transformation
exception class
from tf2_ros.buffer import Buffer                        # Buffer class for storing the
coordinate transformation information
from tf2_ros.transform_listener import TransformListener  # Listener class for monitoring
the coordinate transformations
from geometry_msgs.msg import Twist                      # ROS2 speed control message
from turtlesim.srv import Spawn                          # Turtle spawning service
interface
class TurtleFollowing(Node):

    def __init__(self, name):
        super().__init__(name)                                  # Initialize the parent
class of the ROS2 node
```

```
        self.declare_parameter('source_frame', 'turtle1')           # Create a parameter for
the source coordinate frame name
        self.source_frame = self.get_parameter(                     # The externally set
parameter value takes precedence, otherwise the default value is used
            'source_frame').get_parameter_value().string_value

        self.tf_buffer = Buffer()                                   # Create a buffer to
save the coordinate transformation information
        self.tf_listener = TransformListener(self.tf_buffer, self)  # Create the listener of
the coordinate transformation

        self.spawner = self.create_client(Spawn, 'spawn')           # Create a client that
requests the turtle spawning
        self.turtle_spawning_service_ready = False                  # Flag indicating
whether the turtle spawning service has been requested
        self.turtle_spawned = False                                 # Flag indicating
whether the turtle has been spawned successfully

        self.publisher = self.create_publisher(Twist, 'turtle2/cmd_vel', 1) # Create the
speed topic for the turtle that follows the movement

        self.timer = self.create_timer(1.0, self.on_timer)         # Create a fixed-period
timer to control the movement of the following turtle

    def on_timer(self):
        from_frame_rel = self.source_frame                          # Source coordinate
system
        to_frame_rel   = 'turtle2'                                  # Target coordinate
system

        if self.turtle_spawning_service_ready:                      # If the turtle spawning
service has already been requested
            if self.turtle_spawned:                                 # If the following turtle
has already been spawned
                try:
                    now = rclpy.time.Time()                         # Get the current time of
the ROS system
                    trans = self.tf_buffer.lookup_transform(       # Monitor the coordinate
transformation from the source coordinate system to the target coordinate system at the
current moment
                        to_frame_rel,
                        from_frame_rel,
                        now)
                except TransformException as ex:                    # If the coordinate
transformation acquisition fails, enter the exception report
                    self.get_logger().info(
                        f'Could not transform {to_frame_rel} to {from_frame_rel}: {ex}')
                    return

                msg = Twist()                                       # Create the speed
control message
                scale_rotation_rate = 1.0                           # Calculate the angular
velocity according to the turtle angle
```

```
                msg.angular.z = scale_rotation_rate * math.atan2(
                    trans.transform.translation.y,
                    trans.transform.translation.x)

                scale_forward_speed = 0.5                           # Calculate the linear
velocity according to the turtle distance
                msg.linear.x = scale_forward_speed * math.sqrt(
                    trans.transform.translation.x ** 2 +
                    trans.transform.translation.y ** 2)

                self.publisher.publish(msg)                         # Publish the speed
command, and the turtle follows the movement
            else:                                                   # If the following turtle
has not been spawned
                if self.result.done():                              # Check whether the
turtle has been spawned
                    self.get_logger().info(
                        f'Successfully spawned {self.result.result().name}')
                    self.turtle_spawned = True
                else:                                               # The following turtle is
still not spawned
                    self.get_logger().info('Spawn is not finished')
        else:                                                       # If the turtle spawning
service has not been requested
            if self.spawner.service_is_ready():                     # If the turtle spawning
server is already ready
                request = Spawn.Request()                           # Create the request data
                request.name = 'turtle2'                            # Set the content of the
request data, including the turtle name, the xy position and the posture
                request.x = float(4)
                request.y = float(2)
                request.theta = float(0)

                self.result = self.spawner.call_async(request)      # Send the service
request
                self.turtle_spawning_service_ready = True           # Set the flag to
indicate that the request has been sent
            else:
                self.get_logger().info('Service is not ready')      # Prompt that the turtle
spawning server is not ready yet


def main(args=None):
    rclpy.init(args=args)                           # Initialize the ROS2 Python interface
    node = TurtleFollowing("turtle_following")      # Create and initialize the ROS2 node object
    rclpy.spin(node)                                # Loop and wait for ROS2 to exit
    node.destroy_node()                             # Destroy the node object
    rclpy.shutdown()                                # Shut down the ROS2 Python interface
```

3. Create a new launch folder under the pkg_tf function package, create a file named [turtle_following.launch.py] inside the launch folder, and add the following content:

```
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node

def generate_launch_description():

    return LaunchDescription([
        DeclareLaunchArgument('source_frame', default_value='turtle1', description='Target
frame name.'),
        Node(
            package='turtlesim',
            executable='turtlesim_node',
        ),
        Node(
            package='pkg_tf',
            executable='turtle_tf_broadcaster',
            name='broadcaster1',
            parameters=[
                {'turtlename': 'turtle1'}
            ]
        ),
        Node(
            package='pkg_tf',
            executable='turtle_tf_broadcaster',
            name='broadcaster2',
            parameters=[
                {'turtlename': 'turtle2'}
            ]
        ),
        Node(
            package='pkg_tf',
            executable='turtle_following',
            name='listener',
            parameters=[
                {'source_frame': LaunchConfiguration('source_frame')}
            ]
        ),
    ])
```

8. Editing the configuration file

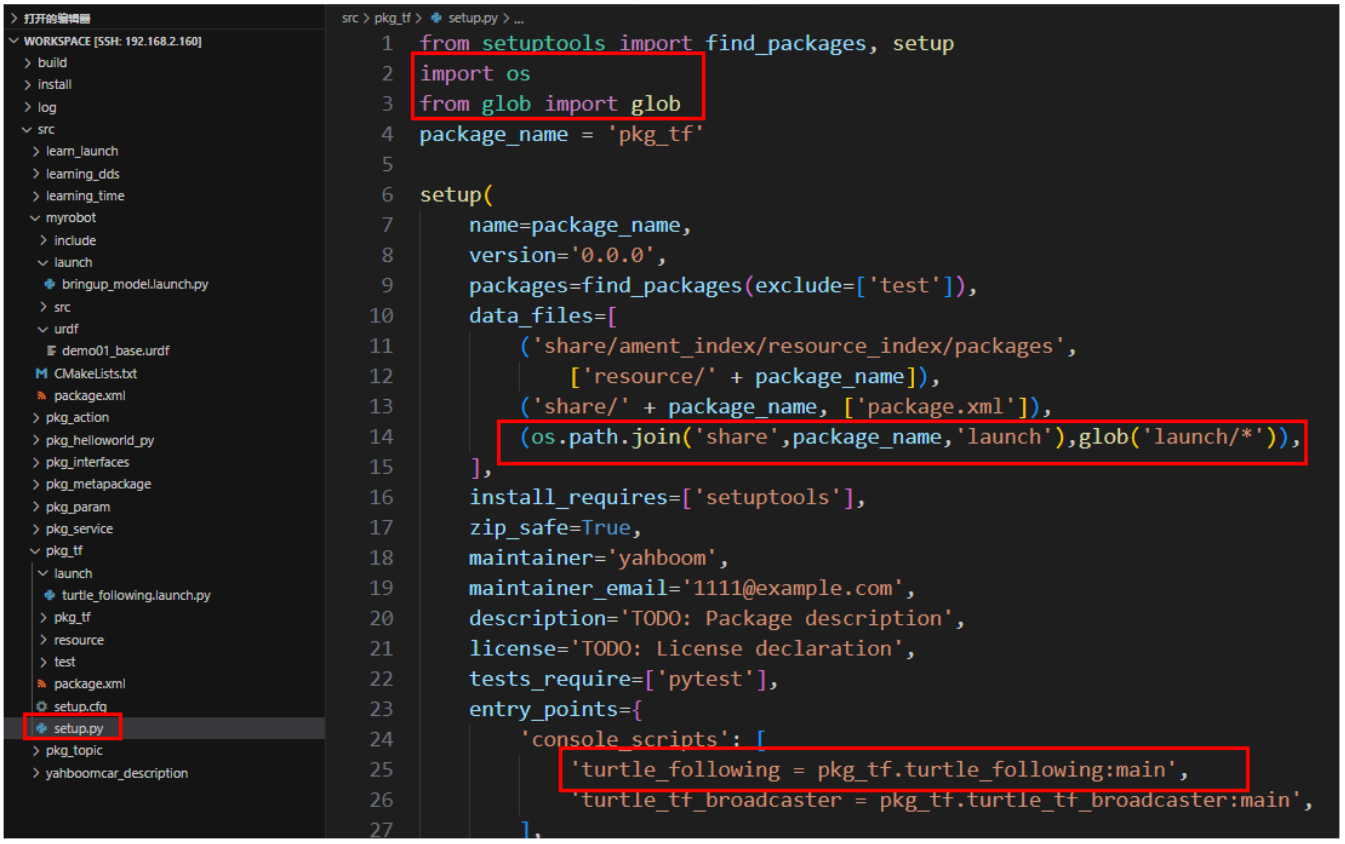
8.1. Configuration in setup.py

- Import the relevant libraries

```
import os
from glob import glob
```

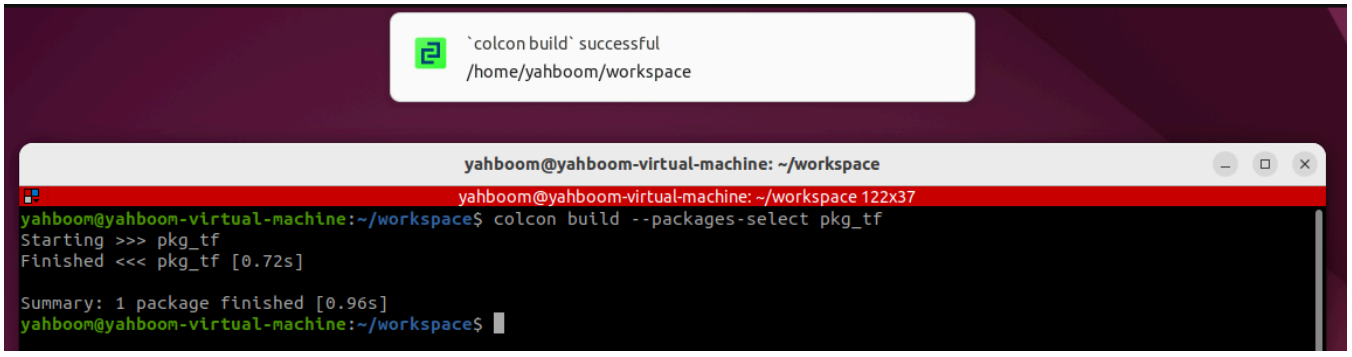
- Add the turtle_following node information, and add the following command to copy the launch file into the shared directory in install

```
(os.path.join('share',package_name,'launch'),glob('launch/*')),
```



9. Compiling the function package

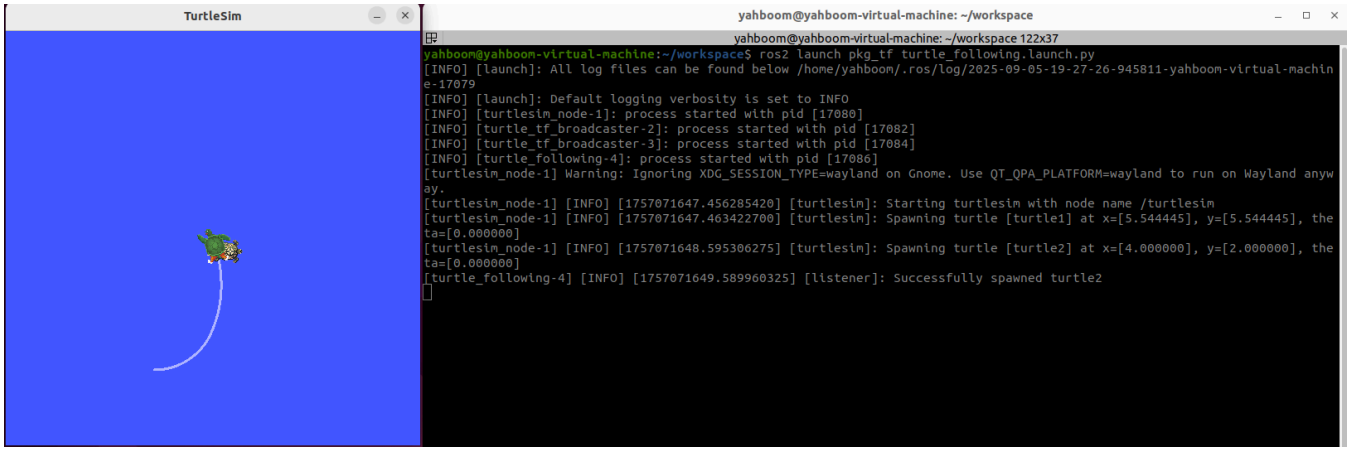
```
colcon build --packages-select pkg_tf
```



10. Running the program

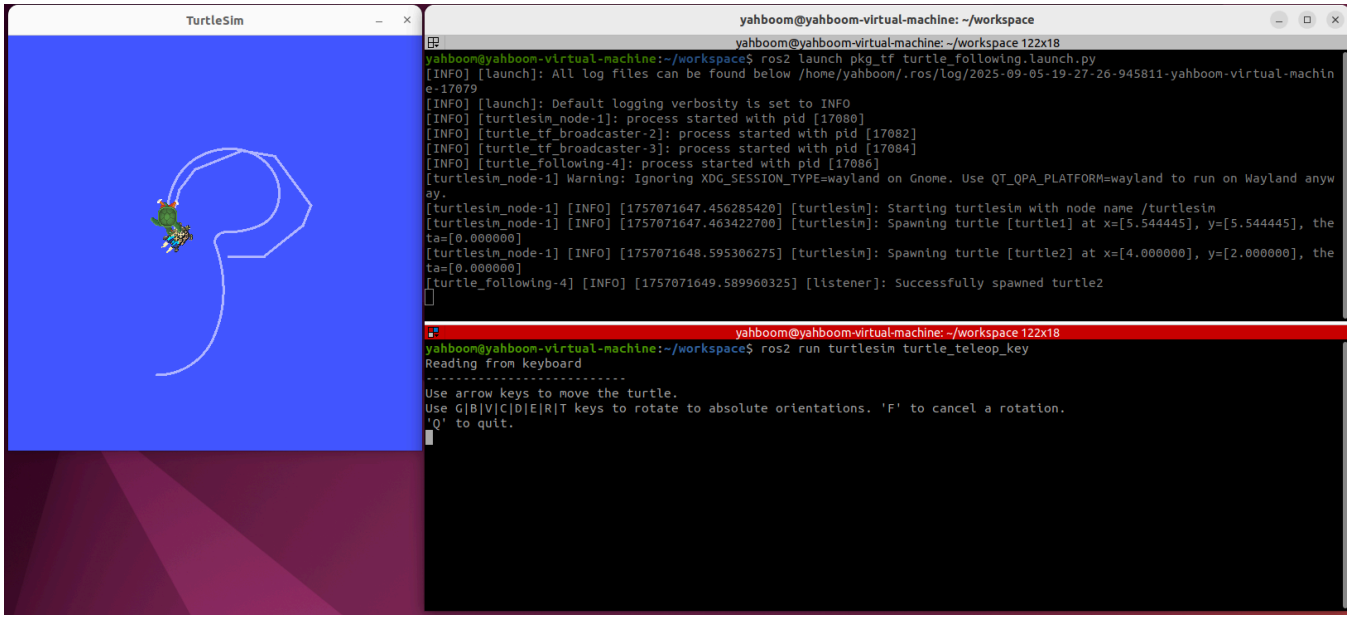
- Refresh the terminal environment variables, then run

```
ros2 launch pkg_tf turtle_following.launch.py
```



- Start the turtle keyboard control node to control the movement of the first little turtle; the second turtle will automatically follow

```
ros2 run turtlesim turtle_teleop_key
```



In this terminal, press the up, down, left and right keys on the keyboard to control the movement of one of the little turtles, and then the other little turtle will follow it until they overlap.

11. Advanced content

- Understanding the cross-time dimension transformation capability of TF

  Buffer can automatically cache all the TF transformation relationships in the TF system over the past 10s through the buffer (you can set any cache duration yourself with the Buffer constructor parameter). All the transformations in the buffer are based on timestamps, and all of them are continuous and traceable. Even if the two coordinate systems are at different points in time, the coordinate transformation relationship between them can still be found. For this point you can refer to the references of designing the TF system, where the principle is explained in detail (the documents are in the folder of this lesson, or you can use the link at the beginning of this lesson)

```
self.tf_buffer = Buffer()                                  # 创建保存坐标变换信息的缓冲区
self.tf_listener = TransformListener(self.tf_buffer, self)  # 创建坐标变换的监听器
```

- The following is a supplement to the turtle following case above. The red arrow indicates that the transformation between two coordinate systems at different points in time can be found across time

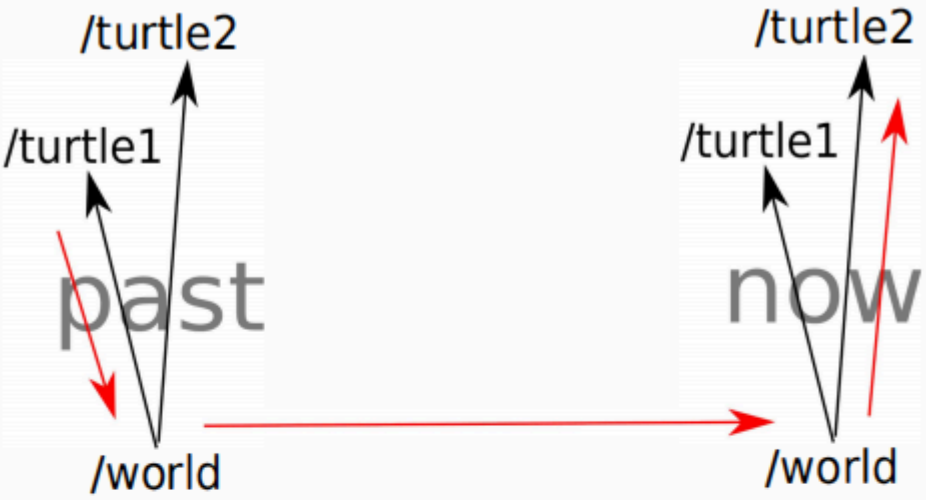


Fig. 4: A simple *tf* tree from a core ROS tutorial, with debugging information.

Long Term Data Storage: The above example assumes that you have the ability to transform between frames a and b at time 0 and b and c at time 1. This will only work if the length of the buffer in your *Listener* module is long enough to encompass both time 0 and time 1. To be able to store data for a long time, it should be transformed into the fixed frame, b, when saved. Then the source frame is the same as frame b and consequently the transform between the two is the identity, leaving only $T_{b@t_1}^{c@t_1}$ needing to be computed to find the current location of any previously observed object. This simple case is common in many robotic systems because they do not have the tools to compute the more complicated cases. This will only work when the assumptions about where the identity transform can be used is maintained.